

# PAGE OBJECT MODEL (POM)

## What is POM?

**POM (Page Object Model)** is a design pattern used in Selenium Automation Frameworks to separate the web element locators from the test scripts.

Instead of directly writing locators inside test cases, all page elements are stored in separate Java classes called **Page Classes**.

### Definition

Page Object Model (POM) is a design pattern that creates an Object Repository for web elements and keeps test scripts independent from UI locators.

## Why Do We Need POM?

### Without POM

```
driver.findElement(By.id("username")).sendKeys("admin");  
driver.findElement(By.id("password")).sendKeys("admin123");  
driver.findElement(By.id("loginBtn")).click();
```

### Problems

- ✗ Hardcoded locators
- ✗ Difficult maintenance
- ✗ Code duplication
- ✗ Poor readability
- ✗ More effort when UI changes

---

### With POM

```
LoginPage lp = new LoginPage(driver);
```

```
lp.enterUsername("admin");  
lp.enterPassword("admin123");  
lp.clickLogin();
```

### Benefits

- ✓ Clean code
- ✓ Easy maintenance
- ✓ Reusable methods
- ✓ Better readability
- ✓ Less code duplication

## Advantages of POM

1. Avoid hardcoding of locators
2. Maintenance of locators is easy
3. Modification of locators is easy
4. Reusability of the code
5. Readability of the code
6. We can handle StaleElementReferenceException.
7. AutoHealing is possible

## Rules of POM

### Rule 1

Number of Web Pages = Number of POM Classes

| Web Page   | POM Class |
|------------|-----------|
| Login Page | LoginPage |
| Home Page  | HomePage  |
| Cart Page  | CartPage  |

### Rule 2

Number of Elements = Number of WebElement Variables

```
@FindBy(id="username")
private WebElement username;
```

### Rule 3

Declare WebElements as Private

```
private WebElement username;
```

### Rule 4

Access Through Public Methods

```
public void enterUsername(String user)
{
    username.sendKeys(user);
}
```

### Rule 5

Initialize Using PageFactory

```
PageFactory.initElements(driver, this);
```

# Steps to Create POM Class

## Step 1

Create Page Class

```
public class LoginPage
{
}
```

---

## Step 2

Store Web Elements

```
@FindBy(id="username")
private WebElement username;

@FindBy(id="password")
private WebElement password;

@FindBy(id="loginBtn")
private WebElement loginBtn;
```

---

## Step 3

Initialize Elements

```
PageFactory.initElements(driver, this);
```

---

## Step 4

Create Constructor

```
public LoginPage(WebDriver driver)
{
    PageFactory.initElements(driver, this);
}
```

---

## Step 5

Create Business Methods

```
public void enterUsername(String user)
{
    username.sendKeys(user);
}

public void enterPassword(String pass)
{
    password.sendKeys(pass);
}

public void clickLogin()
{
    loginBtn.click();
}
```

---

# Complete LoginPage Example

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;
```

```
public class LoginPage
{
    @FindBy(id="username")
    private WebElement username;

    @FindBy(id="password")
    private WebElement password;

    @FindBy(id="loginBtn")
    private WebElement loginBtn;

    public LoginPage(WebDriver driver)
    {
        PageFactory.initElements(driver, this);
    }

    public void enterUsername(String user)
    {
        username.sendKeys(user);
    }

    public void enterPassword(String pass)
    {
        password.sendKeys(pass);
    }

    public void clickLogin()
    {
        loginBtn.click();
    }

    public void login(String user,String pass)
    {
        username.sendKeys(user);
        password.sendKeys(pass);
        loginBtn.click();
    }
}
```

---

# What is PageFactory?

## Definition

PageFactory is a Selenium class used to initialize web elements declared using @FindBy annotation.

Without PageFactory:

```
@FindBy(id="username")
WebElement username;
```

will remain NULL.

When accessed:

```
username.sendKeys("admin");
```

It throws:

NullPointerException

---

## Syntax

```
PageFactory.initElements(driver, this);
```

### Parameters

1. Driver Reference

driver

2. Current Class Object

this

---

## Common Annotations in PageFactory

### @FindBy

Locate single element.

```
@FindBy(id="username")
private WebElement username;
```

---

### @FindBys

AND condition

```
@FindBys({
    @FindBy(tagName="input"),
    @FindBy(id="username")
})
private WebElement username;
```

## @FindAll

OR condition

```
@FindAll({
@FindBy(id="username"),
@FindBy(name="username")
})
private WebElement username;
```

---

## Difference Between POM and PageFactory

| POM                      | PageFactory                  |
|--------------------------|------------------------------|
| Design Pattern           | Selenium Class               |
| Creates Page Objects     | Initializes Elements         |
| Organizes Framework      | Supports POM                 |
| Improves Maintainability | Removes NullPointerException |
| Independent Concept      | Part of POM                  |

### Interview Answer

POM is a design pattern used to separate web elements and business logic from test scripts. PageFactory is a Selenium class used inside POM to initialize the web elements created using @FindBy annotations. We can say PageFactory is a subset of POM.

---

## Drawback of @FindBy

### Dynamic XPath Not Supported

Cannot do:

```
@FindBy(xpath="//td[text()=''+name+'"]")
```

Because values change dynamically.

---

### Solution

Use:

```
driver.findElement(
By.xpath("//td[text()=''+name+'"]")
);
```

or

```
driver.findElements()
```

# Interview Questions

## Q1. What is POM?

A design pattern that stores web elements in separate page classes and keeps test scripts independent of locators.

---

## Q2. Why is POM used?

To improve maintainability, readability, reusability, and reduce code duplication.

---

## Q3. What is PageFactory?

A Selenium class used to initialize elements declared using @FindBy annotations.

---

## Q4. Why use PageFactory.initElements()?

To initialize WebElements and avoid NullPointerException.

---

## Q5. Difference between POM and PageFactory?

POM is a framework design pattern; PageFactory is a Selenium utility class used within POM.

---

## Q6. What are business methods?

Methods that perform user actions.

Example:

```
login();  
logout();  
addToCart();  
createCustomer();
```